Compositional Tracking on Shipping Networks

Extracted from a machine-checked development

Abstract

Given a directed graph G and a monoid M, a *compositional tracking* of M-valued data on the shipping network of G assigns to every route an element of M, sending the empty route to the unit and sending concatenation of routes to multiplication in M. We prove that such trackings are in bijection with arbitrary labellings of the edges of G by elements of M: restriction to single-edge routes and folding of edge labels along a path are mutually inverse. Three monoid instances illustrate the construction—peak temperature, cold-chain integrity, and licence accumulation—and a minimal example shows that a single warm last-mile leg breaks an entire cold-chain route. The proofs of all stated results have been machine-checked; we record only the mathematical content.

Contents

1 Introduction **1**

2 Monoids **2**

3 Compositional trackings **2**

4 Folding along paths **2**

5 From edge labellings to trackings and back **3**

6 Monoid instances **4**

7 Discussion **4**

1 Introduction

Shipping networks, presented as directed containers, organise nodes and directed paths. Once that geometry is fixed, one often wishes to track *data along routes*: peak temperatures, cold-chain integrity flags, accumulated licence requirements, and so on. Compositionality demands that the value on a concatenated route be the monoid product of the values on the factors, and that the empty route carry the monoid unit.

The present note isolates that structure. Working directly on the directed container network(G) associated to a directed graph G, a compositional tracking with values in a monoid M is a family of maps from positions to M satisfying a root law and a shift law. The main theorem is a bijection: every tracking arises uniquely by folding an edge labelling along paths, and every edge labelling extends uniquely to a tracking.

Categorically, a tracking is a functor from the free category on G to the delooping of M (the one-object category with morphisms M). The edge-folding description is the elementary content of that functoriality.

The development works throughout in a single universe of types. Proofs are indicated by the phrase "by lean," meaning they have been checked by a proof assistant rather than written out in prose.

What this note claims. In ordinary mathematical language: monoid-valued functors on the free category of a directed graph are precisely edge labellings, extended by multiplication along paths. There is no further coherence data beyond the monoid laws of M and the path algebra of G.

2 Monoids

Definition 2.1 (Monoid)**.** A **monoid** M consists of a type $|M|$, a binary operation $\cdot : |M| \to |M| \to |M|$, and a unit element $1 \in |M|$, such that for all $a, b, c \in |M|$,

$$(a \cdot b) \cdot c \;=\; a \cdot (b \cdot c), \qquad 1 \cdot a \;=\; a, \qquad a \cdot 1 \;=\; a. \tag{1}$$

We work with monoids as bundled structures rather than as typeclass instances. When the monoid is clear we write M both for the structure and for its carrier.

3 Compositional trackings

Fix a directed graph G with node type V and edge type E, and write $\mathcal{N} := \mathrm{network}(G)$ for the associated directed container: shapes are nodes; a position at v is a pair $\langle w, \rho \rangle$ of an endpoint and a finite directed path ρ from v to w; the root is the empty path; sub returns the endpoint; and shift is path concatenation.

Definition 3.1 (Compositional tracking)**.** Let M be a monoid. A **compositional tracking** of M-data on $\mathcal{N}$ consists of a family

$$d \;:\; \prod_{v\,:\,V} \Big(\mathrm{Pos}(\mathcal{N})(v) \;\longrightarrow\; M\Big) \tag{2}$$

satisfying

1. **(T-root)** for every node v,
$$d_v(\mathrm{root}(v)) \;=\; 1;$$
2. **(T-shift)** for every node v, every position $\rho \in \mathrm{Pos}(\mathcal{N})(v)$, and every position $\sigma \in \mathrm{Pos}(\mathcal{N})(\mathrm{sub}(v, \rho))$,
$$d_v(\mathrm{shift}(v, \rho, \sigma)) \;=\; d_v(\rho) \cdot d_{\mathrm{sub}(v,\rho)}(\sigma).$$

Thus empty routes are labelled by the unit, and concatenated routes are labelled by monoid products. Equivalently, d is a functor $\mathcal{N}(G) \to BM$ from the free category on G to the delooping of M.

4 Folding along paths

Definition 4.1 (Path fold)**.** Let $f : E \to M$ be a function labelling edges by monoid elements. The **fold** of f along a path is the map

$$\mathrm{fold}_f \;:\; \prod_{v,w\,:\,V} (\mathrm{Path}_G(v, w) \;\longrightarrow\; M) \tag{3}$$

defined by

$$\mathrm{fold}_f(\mathrm{nil}(v)) \;=\; 1, \tag{4}$$
$$\mathrm{fold}_f(\mathrm{cons}(e,p)) \;=\; f(e)\cdot \mathrm{fold}_f(p). \tag{5}$$

Proposition 4.2 (Fold of the empty path)**.** *For every edge labelling $f : E \to M$ and every node v,*

$$\mathrm{fold}_f(\mathrm{nil}(v)) \;=\; 1.$$

Proof. By lean. □

Proposition 4.3 (Fold preserves concatenation)**.** *For every edge labelling $f : E \to M$ and paths $p : \mathrm{Path}_G(v,w)$, $q : \mathrm{Path}_G(w,u)$,*

$$\mathrm{fold}_f(p +\!\!+ q) \;=\; \mathrm{fold}_f(p)\cdot \mathrm{fold}_f(q).$$

Proof. By lean. □

The empty-path case is immediate from the left unit law of M. The inductive step for a cons-path uses associativity of multiplication.

5 From edge labellings to trackings and back

Definition 5.1 (Tracking from an edge labelling)**.** Let $f : E \to M$. Define a tracking $\mathrm{ofEdge}(f)$ by

$$d_v\langle w,\rho\rangle \;=\; \mathrm{fold}_f(\rho). \tag{6}$$

The root law holds by Proposition 4.2; the shift law holds by Proposition 4.3.

Definition 5.2 (Single-edge position)**.** For an edge $e \in E$, the **single-edge position** at $\mathrm{src}(e)$ is

$$\varepsilon_e \;=\; \langle \mathrm{tgt}(e),\, \mathrm{cons}(e,\, \mathrm{nil}(\mathrm{tgt}(e)))\rangle \;\in\; \mathrm{Pos}(\mathcal{N})(\mathrm{src}(e)). \tag{7}$$

Definition 5.3 (Edge labelling from a tracking)**.** Let t be a tracking with value family d. Define

$$\mathrm{toEdge}(t) \;:\; E \;\longrightarrow\; M, \qquad e \;\longmapsto\; d_{\mathrm{src}(e)}(\varepsilon_e). \tag{8}$$

Lemma 5.4 (Fold recovers the tracking)**.** *Let t be a tracking with value family d. For every path $p : \mathrm{Path}_G(v,w)$,*

$$\mathrm{fold}_{\mathrm{toEdge}(t)}(p) \;=\; d_v\langle w,\, p\rangle. \tag{9}$$

Proof. By lean. □

Remark 5.5 (Idea of the argument)**.** The proof is by induction on p. The nil case is the root law. For a cons-path $\mathrm{cons}(e,p)$, the shift law applied to the single-edge position ε_e and the residual position $\langle w,p\rangle$ yields the multiplicative step, after observing that shifting ε_e by that residual is definitionally $\langle w,\, \mathrm{cons}(e,p)\rangle$.

Theorem 5.6 (Round-trip: tracking to edges and back)**.** *For every tracking t,*

$$\mathrm{ofEdge}(\mathrm{toEdge}(t)) \;=\; t.$$

Proof. By lean. □

Theorem 5.7 (Round-trip: edges to tracking and back)**.** *For every edge labelling $f : E \to M$,*

$$\mathrm{toEdge}(\mathrm{ofEdge}(f)) \;=\; f.$$

Proof. By lean. □

Theorem 5.8 (Tracking–edge bijection)**.** *For every directed graph G and monoid M, there is a bijection*

$$\{\, \textit{compositional trackings of } M\textit{-data on } \mathrm{network}(G) \,\} \;\cong\; \{\, \textit{functions } E \to M \,\}, \tag{10}$$

implemented by toEdge *in one direction and* ofEdge *in the other. The two constructions are mutually inverse by Theorems 5.6 and 5.7.*

Proof. By lean. □

6 Monoid instances

Example 6.1 (Peak temperature)**.** Let MaxTemp = {Cold, Warm} with multiplication

$$\mathrm{Cold} \cdot x \;=\; x, \qquad \mathrm{Warm} \cdot x \;=\; \mathrm{Warm}, \tag{11}$$

and unit Cold. This is an idempotent monoid: a single warm leg along a route dominates all cold legs, and subsequent refrigeration cannot undo a prior breach of the temperature bound.

Example 6.2 (Cold-chain integrity)**.** Let $M = \{\mathsf{true}, \mathsf{false}\}$ with multiplication given by boolean conjunction and unit true. Interpreting true as "intact" and false as "broken," a route is intact if and only if every leg is intact. Once a leg breaks the chain, no subsequent leg can repair it.

Example 6.3 (Licence accumulation)**.** Let L be a type of licence tokens, and take the monoid of finite lists of elements of L under concatenation, with unit the empty list. Routes accumulate the licences required by their individual legs. (Deduplication is not needed for the algebraic structure.)

Proposition 6.4 (Warm last-mile breaks the cold chain)**.** *Consider the two-node digraph with nodes* {Hub, Shop} *and a single edge* lastMile : Hub → Shop*, labelled* false *in the cold-chain monoid of Example 6.2. Let t be the tracking induced by this edge labelling. Then:*

1. *the empty route at* Hub *is intact (t returns* true*);*
2. *the single-edge route* Hub → Shop *is broken (t returns* false*).*

Proof. By lean. □

7 Discussion

What has been established. Compositional trackings on a shipping network are completely classified by edge labellings with values in the target monoid. The reconstruction is canonical: fold labels along paths in one direction, restrict to single-edge routes in the other. The monoid laws of M and the path algebra of G are the only ingredients.

Relation to functors into deloopings. The root and shift laws are exactly the requirements that d preserve identities and composition in the free category on G. The edge-labelling description is the usual fact that a functor out of a free category is determined by its values on generating morphisms.

Scope. The development treats a single monoid of values and does not address multi-parameter trackings, dependent value types, or interaction with aggregation morphisms of networks. Those extensions sit on top of the same bijection.

Machine checking. Every definition and theorem statement above corresponds to a declaration in a machine-checked formalisation. The phrase "by lean" indicates that the proof has been verified by a proof assistant.

Acknowledgements

The mathematical content follows the construction of compositional trackings in the supply-chain development that motivated this note. The presentation is a prose rendering of a formal development intended for human review of the theorem statements.

References

[1] M. Abbott, T. Altenkirch, and N. Ghani. Containers: constructing strictly positive types. *Theoretical Computer Science*, 342(1):3–27, 2005.

[2] D. Ahman and T. Uustalu. Coalgebraic update lenses. In *MFPS*, volume 308 of *ENTCS*, pages 25–48, 2014.